

Week 1: Software arch.

Week 18

What is Software Architecture?

- It is the princ. of design decisions.
- > set of struc. need a reason about the sys. for: long process & huge cost.
- Documentation is critical

It provides: $\Rightarrow$ high quality software.

- Notes: - It is an abstraction - about quality - It is not a step, it starts from the beginning

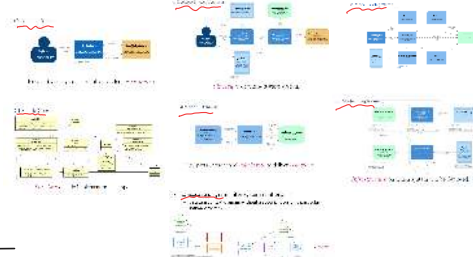
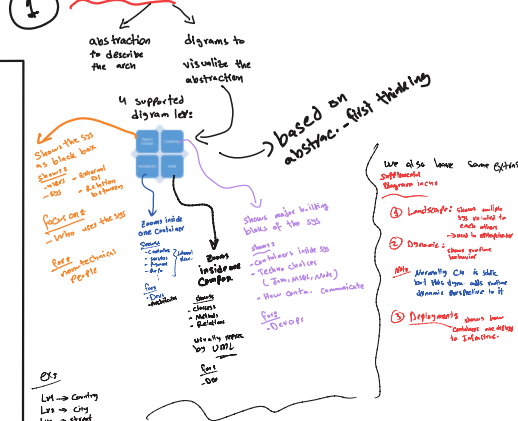
Arch. degradation

What makes a good arch.?

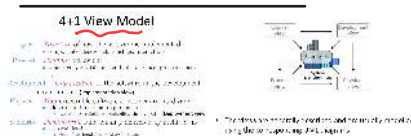
$\Rightarrow$ there is no good or bad, arch. are either more fit for specific purpose and context

arch. views

View 1: C4 models 2 basic Concept:

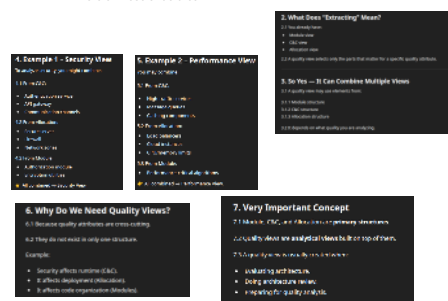


View 2



Quality view

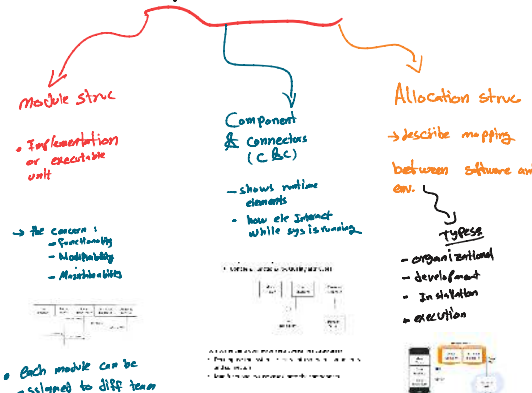
It is formed by extracting relevant parts from one or more of those structures.



Combined view:



Software Arch. in practice views



Notes

Modules = Compile time
Component = Runtime

- Modules are built into components

Quality attr.:

Availability: -> about is sys. working?
Fundamentally, availability is about minimizing service outage time by mitigating faults.
The goal is make the time sys. not working as short as possible (min. duration)

Week 23

Basic Properties of Styles

- Operational categories:**
 - Availability
 - Reliability
 - Scalability
 - Performance
 - Deployability
 - Scalability
 - Interoperability
 - Compliance
 - Security
- Developmental categories:**
 - Identifiability
 - Verifiability
 - Supportability
 - Testability
 - Maintainability
 - Portability
 - Flexibility
 - Development distribution
 - Reusability

Benefits of styles:

Recaps

Lecture 1 — Why Software Architecture Matters

Hardware: Hardware is the physical infrastructure that the software runs on. It includes the CPU, memory, and storage.

Software: Software is the set of instructions that tell the hardware what to do. It includes the operating system, applications, and data.

Very simple example: A simple web application. The hardware is the server, and the software is the web browser. The server sends data to the browser, and the browser displays the data.

Key takeaways:

- Architecture is the foundation of a system.
- Good architecture leads to better quality software.
- Architecture is a design decision.

Lecture 2 — Architectural Patterns

Patterns are reusable solutions to common problems. They provide a template for designing a system.

Examples of patterns include the Singleton, Factory, and Observer patterns.

Key takeaways:

- Patterns are a way to organize code.
- Patterns can make code easier to understand.
- Patterns can help you avoid common mistakes.

Architectural Patterns

- An architectural pattern is:
 - a set of architectural design decisions
 - that are applicable to a recurring design problem
 - and parameterized to account for different software development contexts in which that problem appears.

What is a Pattern?

Structure of a Pattern

According to the lecture, every architectural pattern defines the following:

- Context:** The situation where the problem occurs.
- Problem:** The recurring challenge or problem.
- Solution:** The reusable solution to the problem.

The goal is to find a solution that works for a specific problem.

Why Architectural Patterns Are Important

They help achieve system qualities such as:

Quality Attribute	Impact
Scalability	handle growth
Maintainability	easier evolution
Reliability	stable structure
Performance	efficient operation

These qualities are often the main reason for selecting a pattern.

- Difference between architectural patterns and software design patterns?

Architectural Patterns vs Design Patterns

Architectural patterns define the overall structure of a system, while design patterns define the structure of individual components.

Examples of architectural patterns include the Client-Server, Peer-to-Peer, and Layered patterns.

Examples of design patterns include the Singleton, Factory, and Observer patterns.

with Notes

Reviewing

Performances

Scalability:

Scalability is a property of a system that allows it to handle an increasing amount of work or to expand its capacity in order to meet the growing demands of its users.

→ about if sys is working correctly?

→ about how fast the sys is?

→ about where workload is high, but the sys works same. (Same perf time)

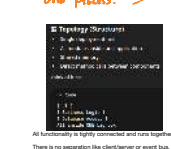
What is an arch. style?

An architectural style is a named way of organizing a software system. It gives rules about how parts connect and work together, and it helps achieve certain quality goals.

1) Monolithic Arch.

system where all parts of the app are built and deployed as one single unit.

→ one codebase, one deployment, one process.



Advantages:

- Easy to develop
- Easy to deploy
- Simple arch.

Disadvantages:

- High coupling (everything is coupled)
- Hard to test
- Scale is a challenge
- Single point of failure

Quality attributes	Score
Flexibility	Low
Scalability	Low
Reliability	Low
Performance	Low
Security	Low
Testability	Low

When is it Suitable?

- Small systems
- Simple applications
- Early prototypes
- Student projects

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

Important Exam Idea

Monolithic architecture is a single unit that is difficult to scale and maintain.

2) Layered Arch.

is a sys divided into sep. layers where each layer has a specific function and communicates only with its neighboring layers.

→ each layer builds on top of the one below it.



Advantages:

- Abstraction
- Reuse
- Clear separation of concerns
- Easy to maintain
- Scalability
- Testability

Disadvantages:

- Performance overhead
- May not suit modern infrastructures
- Some security issues
- Multiple data centers
- Error prone updates

Quality attributes	Score
Flexibility	High
Scalability	High
Reliability	High
Performance	Low
Security	Low
Testability	High

When is it Suitable?

- Enterprise systems
- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

3) Client-server

is a sys where clients req. services and server provides services. It manages data over a network.

→ can be called 2-tier.

→ client and server are sep. and often on diff. machines.



Advantages:

- High availability
- Easy to scale
- Centralized data management
- Security
- Easy to update

Disadvantages:

- Not suitable for interactive apps
- Not good for mobile devices
- Network latency affects perf.
- Reg. network 2factive.

Quality attributes	Score
Flexibility	Low
Scalability	High
Reliability	High
Performance	Low
Security	High
Testability	Low

When is it Suitable?

- Enterprise systems
- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

Important Exam Idea

Client-server architecture is a two-tier system where clients request services from a central server.

4) Pipe and Filter

is a style where data flows through a series of independent components (filters), connected by pipes.

→ each filter receives input data, processes it, and produces output data.



Advantages:

- High modularity
- Easy to scale
- Fault tolerance
- Easy to test

Disadvantages:

- Not suitable for interactive apps
- Not good for mobile devices
- Network latency affects perf.
- Reg. network 2factive.

Quality attributes	Score
Flexibility	High
Scalability	High
Reliability	High
Performance	Low
Security	Low
Testability	High

When is it Suitable?

- Enterprise systems
- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Important Exam Idea

Pipe and filter architecture is a multi-tier system where data flows through a series of filters.

Layered :

- state - logic - display

- Model - view - Controller MVC

Definition

Layered patterns are architectural patterns where the system is divided into layers, each layer has a specific responsibility.

The idea is to separate parts of the system that change at different rates.

Your side says:

- "Separate elements with different rate of change"

Meaning:

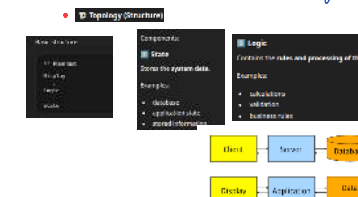
Some parts of a system change often (like UI), while others change rarely (like database).

Layered patterns separate these parts to make the system easier to maintain.

1. State - logic

→ separates the sys into

- state - logic



Main Idea

Different parts of software change at different speeds.

Example:

Component	Change Frequency
Display	changes very often
Logic	changes sometimes
State	changes rarely

So the architecture separates them.

Advantages

- Modularity
- Reuse
- Testability
- Scalability

Disadvantages

- Performance
- Complexity
- Maintenance

Suitable Applications

- UI changes frequently
- Logic should remain stable
- Data must be protected

Examples:

- desktop applications
- early web systems
- interactive systems

Quality attributes	Score
Flexibility	High
Scalability	High
Reliability	High
Performance	Low
Security	Low
Testability	High

When is it Suitable?

- Enterprise systems
- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Important Exam Idea

Layered architecture is a multi-tier system where each layer has a specific function.

Because systems often contain many independent services that must work together,

1 - interoperability

→ it enables communication between technologies, platforms, or products



- **Practicality**
 - clear standard terminology or protocols, idioms, or "best practice"
- **Extensibility**
 - how easy are things to modify
 - $100\% = \text{no use at all}$ (e.g. $50\% = \text{half}$)



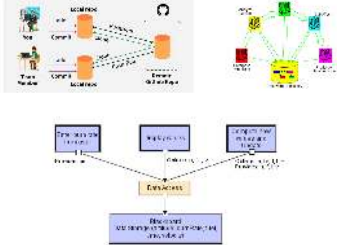
- | Advantages | Disadvantages |
|-------------------------|-------------------------------|
| Technology independence | additional complexity |
| Integration support | latency |
| Flexibility | protocol compatibility issues |
| Scalability support | |

- [illegible]

- loose coupling between processing components.



Repository Style

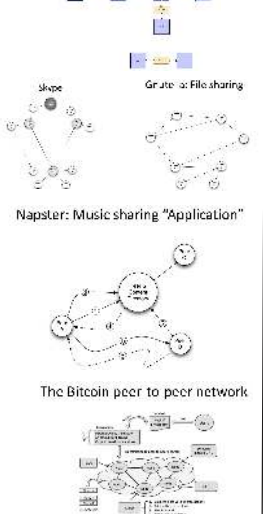


- Support Product Variability



② Peer-to-Peer (P2P)

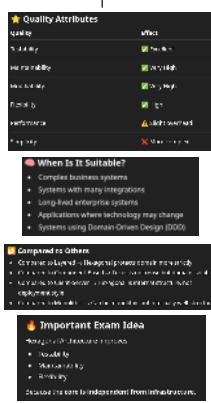
- is a style where all nodes are equal and each node can act as both Client and



Week 3:

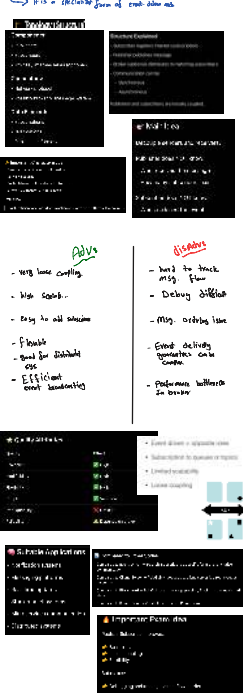
Adv. | Disadv.

- | | |
|---------------------------------|---------------------------|
| - testability | - Complexity |
| - Technology Indep. | - more abstraction layers |
| - strong separation of concerns | - performance oriented |
| - good maintain. | - Overkill for small sys |
| - flexibility | |



⑩ Publish - Subscribe

- is a pattern where publishers send messages without knowing who receives them and subscribers receive messages without knowing who sent them.

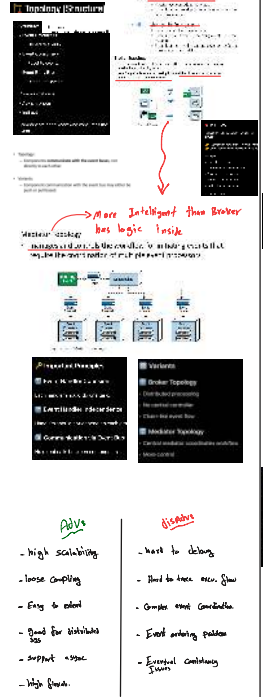


[Twitter](#) - [RSS Feeds](#) - [Email](#)



⑨ Event-Driven

- is a style where Components communicate by producing and Consuming events async through an event bus.



73-10

-
- Introduction**
- Windows 7**
- New Start menu
 - New taskbar
 - New navigation pane
 - New Windows Explorer
 - New Windows Firewall
 - New Windows Defender
 - New Windows Update
- Windows 8**
- New Start screen
 - New taskbar
 - New navigation pane
 - New Windows Explorer
 - New Windows Firewall
 - New Windows Defender
 - New Windows Update
- Comparison to Others**
- | Feature | Windows 7 | Windows 8 |
|------------------|-----------|-----------|
| Start menu | Yes | No |
| Taskbar | Yes | No |
| Navigation pane | Yes | No |
| Windows Explorer | Yes | No |
| Windows Firewall | Yes | No |
| Windows Defender | Yes | No |
| Windows Update | Yes | No |



 Definition

directly.

An **event** is something that happens in the system.

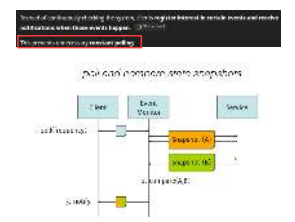
Examples:

- user logs in
- order is placed
- CPU usage exceeds limit
- payment completed

In event-based systems, components **listen for events**

1. Event monitor pat.

- Notifies client when a specific event occurs in a service.



[E Topdeg\(5\) is 100%](#)



Topology (Struct.



Distributed System

- A system with multiple components located on *different machines* that communicate and coordinate actions in order to *appear as a single coherent system* to the end-user.

What are the benefits?

- Improved *reliability*
- Improved *scalability*
- Improved *latency*

What are the drawbacks?

- Increased *complexity*
- Increased *attack surface*
- Increased *latency*
- Introduce *consistency* problems

1 Service oriented Arch

S.O Arch is the composition of software Components (services) to form a Complete Sys.

Service oriented Arch

Service oriented architecture (SOA) is a design pattern in which applications are built from services that communicate over a network.

Key Characteristics:

- Interoperability: Services are built using standard protocols and interfaces.
- Loose coupling: Services are independent and can be developed, deployed, and updated separately.
- Discoverability: Services are discoverable through a central registry or directory.
- Reusability: Services are reusable and can be composed to form new applications.
- Scalability: Services can be scaled independently.
- Flexibility: Services can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- XML: Extensible Markup Language for data exchange.
- SOAP: Simple Object Access Protocol for message exchange.
- WSDL: Web Services Description Language for service discovery.
- UDDI: Universal Description, Discovery, and Integration for service registry.

2 REST

- is an arch. style for distributed sys defining certain constraints that provide efficiency, scal. and other desirable attr.

REST (Representational State Transfer)

REST is an architectural style for designing distributed systems. It is based on a set of constraints that guide the design of the system.

Key Constraints:

- Client-server: The system is composed of a set of clients and servers. Clients interact with servers to retrieve and manipulate data.
- Statelessness: Each request from a client to a server must contain all the information needed to understand the request, and the server must not store any client state.
- Cacheability: Responses must be cacheable to improve performance.
- Interface Uniformity: The interface must be uniform, allowing for a simple and consistent way to interact with the system.
- Layered system: The system can be composed of multiple layers, each responsible for a specific function.
- Code on demand: Optional and can be used to extend the functionality of the system.

Key Characteristics:

- Simple and easy to understand and implement.
- Scalable and flexible.
- Supports a wide range of data formats.
- Supports a wide range of communication protocols.

3 microservices Arch.

- is an evolution of service-oriented Arch (SOA)

Microservices Architecture

Microservices architecture is a design pattern in which applications are built from small, independent services that communicate over a network.

Key Characteristics:

- Small and independent services: Each service is a small, self-contained module that performs a specific function.
- Loose coupling: Services are independent and can be developed, deployed, and updated separately.
- Discoverability: Services are discoverable through a central registry or directory.
- Reusability: Services are reusable and can be composed to form new applications.
- Scalability: Services can be scaled independently.
- Flexibility: Services can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

4 Distributed Monolith (Anti-Pattern)

- The essence of the distributed monolith is lack of isolation between services.

Distributed Monolith (Anti-Pattern)

A distributed monolith is a system that looks like a monolith, but the services are tightly coupled and dependent on each other, just like a monolith system.

Key Characteristics:

- Tight coupling: Services are tightly coupled and dependent on each other.
- Scalability issues: Scaling the system is difficult because all services must be scaled together.
- Deployment complexity: Deploying the system is complex because all services must be deployed together.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

5 Migrating a monolith

- it decompose the monolith into services using bounded contexts and Inter services

Migrating a monolith

Migrating a monolith involves decomposing the monolith into services using bounded contexts and inter-services.

Key Concepts:

- Bounded contexts: Services are designed to have a specific, well-defined scope of responsibility.
- Inter-services: Services are designed to interact with each other in a well-defined way.

Key Challenges:

- Complexity: Migrating a monolith is a complex task that requires a lot of planning and coordination.
- Scalability: Migrating a monolith can be a scalability issue because all services must be scaled together.
- Deployment complexity: Deploying the system is complex because all services must be deployed together.

Key Technologies:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

- Advantages:**
- loose coupling between services
 - Reusability of services
 - Good interoperability
 - Integrated legacy sys.
 - suitable for large enterprise sys.
 - standard interfaces & interfaces
- Disadvantages:**
- ESB problems: central bottleneck
 - hard to scale
 - long syncronous
 - complex setup
 - Business logic moving into ESB
 - Other issues
 - Shared DB makes service hard to manage
 - failure of one service may affect workflow

Service oriented Arch

Service oriented architecture (SOA) is a design pattern in which applications are built from services that communicate over a network.

Key Characteristics:

- Interoperability: Services are built using standard protocols and interfaces.
- Loose coupling: Services are independent and can be developed, deployed, and updated separately.
- Discoverability: Services are discoverable through a central registry or directory.
- Reusability: Services are reusable and can be composed to form new applications.
- Scalability: Services can be scaled independently.
- Flexibility: Services can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- XML: Extensible Markup Language for data exchange.
- SOAP: Simple Object Access Protocol for message exchange.
- WSDL: Web Services Description Language for service discovery.
- UDDI: Universal Description, Discovery, and Integration for service registry.

Web Services (SOAP) - Cloud Computing

Web services are a set of protocols and standards for making distributed applications available over a network.

Key Characteristics:

- Interoperability: Web services are built using standard protocols and interfaces.
- Loose coupling: Web services are independent and can be developed, deployed, and updated separately.
- Discoverability: Web services are discoverable through a central registry or directory.
- Reusability: Web services are reusable and can be composed to form new applications.
- Scalability: Web services can be scaled independently.
- Flexibility: Web services can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- XML: Extensible Markup Language for data exchange.
- SOAP: Simple Object Access Protocol for message exchange.
- WSDL: Web Services Description Language for service discovery.
- UDDI: Universal Description, Discovery, and Integration for service registry.

Advantages from the above presentation

Advantages from the above presentation include:

- Improved reliability
- Improved scalability
- Improved latency
- Increased complexity
- Increased attack surface
- Increased latency
- Introduce consistency problems

Key Technologies

Key technologies for distributed systems include:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

- Advantages:**
- Scalable
 - loose coupling
 - Interoperability
 - Performs improv. via caching
 - Simplicity
- Disadvantages:**
- Statelessless on increase work.
 - Not ideal for complex Transactions.
 - over fetching or under-fetch.
 - Network overhead

Quality Attributes

Quality attributes for distributed systems include:

- Availability: The system is available and accessible.
- Scalability: The system can handle increasing loads.
- Performance: The system performs well under various conditions.
- Reliability: The system is reliable and consistent.
- Security: The system is secure and protected.
- Interoperability: The system can interact with other systems.

World Wide Web - RESTful Web APIs

World Wide Web - RESTful Web APIs are a set of protocols and standards for making distributed applications available over a network.

Key Characteristics:

- Interoperability: RESTful Web APIs are built using standard protocols and interfaces.
- Loose coupling: RESTful Web APIs are independent and can be developed, deployed, and updated separately.
- Discoverability: RESTful Web APIs are discoverable through a central registry or directory.
- Reusability: RESTful Web APIs are reusable and can be composed to form new applications.
- Scalability: RESTful Web APIs can be scaled independently.
- Flexibility: RESTful Web APIs can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- XML: Extensible Markup Language for data exchange.
- SOAP: Simple Object Access Protocol for message exchange.
- WSDL: Web Services Description Language for service discovery.
- UDDI: Universal Description, Discovery, and Integration for service registry.

Slays last part

Fallacies of Distributed Computing

Fallacies of Distributed Computing

Fallacies of distributed computing are common misconceptions about distributed systems.

Key Fallacies:

- The Network is Reliable: The network is not always reliable, and data can be lost or corrupted.
- Network Performance is Always Good: Network performance can be poor, and latency can be high.
- Availability is Unlimited: Availability is limited, and services can be unavailable.
- Security is Not a Problem: Security is a major concern, and distributed systems are vulnerable to attacks.
- Network Topology is Not a Problem: Network topology is a major concern, and distributed systems are vulnerable to network failures.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

- Advantages:**
- Gradual sys modernization
 - Reduce migration risks
 - Continuous sys open
 - Better scale.
 - Improved modularity
- Disadvantages:**
- migration complexity
 - Temporary hybrid arch
 - Increased sys complexity during transition
 - Hard to control service boundary

Key Technologies

Key technologies for distributed systems include:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

Service oriented Arch

Service oriented architecture (SOA) is a design pattern in which applications are built from services that communicate over a network.

Key Characteristics:

- Interoperability: Services are built using standard protocols and interfaces.
- Loose coupling: Services are independent and can be developed, deployed, and updated separately.
- Discoverability: Services are discoverable through a central registry or directory.
- Reusability: Services are reusable and can be composed to form new applications.
- Scalability: Services can be scaled independently.
- Flexibility: Services can be changed or replaced without affecting the entire system.

Key Concepts:

- Service: A self-contained module that performs a specific function.
- Service Interface: A contract that defines the service's capabilities and how to interact with it.
- Service Registry: A central repository that stores information about available services.
- Service Consumer: An application that uses the services provided by the service registry.

Key Technologies:

- XML: Extensible Markup Language for data exchange.
- SOAP: Simple Object Access Protocol for message exchange.
- WSDL: Web Services Description Language for service discovery.
- UDDI: Universal Description, Discovery, and Integration for service registry.

Definition

Composition patterns describe how multiple services perform a task.

These patterns are especially used in:

- distributed systems
- microservices architectures
- large-scale data processing systems

- 1. scatter - Gather**
- Distributes a req. to multiple services in parallel, collects their res., and then combines the res. into one final res.
- 2. load balancing**
- The load balancing service routes requests to different servers to improve performance.

Key Technologies

Key technologies for distributed systems include:

- Containerization: Technologies like Docker and Kubernetes for managing containers.
- Cloud-native: Designing applications to run on cloud infrastructure.
- API Gateways: Central points for managing and routing requests.

Quality Attributes (Highlighted Ones)

Quality attributes for distributed systems include:

- Availability: The system is available and accessible.
- Scalability: The system can handle increasing loads.
- Performance: The system performs well under various conditions.
- Reliability: The system is reliable and consistent.
- Security: The system is secure and protected.
- Interoperability: The system can interact with other systems.

Real-World Example

Real-world examples of distributed systems include:

- Amazon: A large-scale e-commerce platform.
- Google: A large-scale search engine.
- Facebook: A large-scale social media platform.

B - Arch. by Implication

Definition
The architecture by implication will be chosen when a system has no explicit definition.

architects, and developers assume the architecture will emerge automatically during development. Each the actors role:

- Architects is not explicitly defined or documented (developers assume it will evolve naturally as the system grows). ☐ **no control**
- Developers have

Instead of planning architecture, developers say:

"We will figure it out later."

As architecture emerges accidently rather than being designed.

Topology [Structure] Main Problem

Developers will require:	Because architecture is non-definite
2 or 3 Backend Engineers per server	• developers write local decisions • components evolve incrementally • the system eventually becomes hard to maintain
System owners will require:	Over time the system may turn into a Ball of Mud.
There is no architectural change or documentation	

Example

The screenshot shows the Xfce desktop environment. The Xfce Settings window is open, displaying the 'System' tab. The window title is 'Xfce Settings'. The 'System' tab is selected, showing system information: Name (Xfce), Version (4.12.0), and Distribution (Ubuntu 14.04 LTS). The 'Window Management' section is expanded, showing settings for Window List, Taskbar, and Window Buttons. The 'Window List' section is currently active, showing settings for 'Show window list' (checked), 'Show window list icon' (checked), and 'Show window list icon' (checked).

★ Quality Attributes Impact

[illegible]

Suggested Solution (from lecture)

```
The solution is explicit architectural design.

Recommended steps:

1. Define architecture early
2. Encourage architectural decisions
3.
4. Use architectural patterns
5.
6. Reuse architecture regularly

Architecture should be planned and designed by the architectural community.
```

Real-World Example

Startups sometimes fall into this anti-pattern when they prioritize:

- "Ship features fast"
- without designing architecture.

Later, when the system grows, they must refactor the entire system.



- **Problems**
 - Lack of architecture planning and documentation due to architect over confidence or inexperience leads to information on risks.
- **Solution**
 - An organized approach to systems architecture distillars the issues on multiple views of the system

Definition

...the value is

Journal of Management Education

- we tried to develop
- prove it later on
- is the necessary

Negative Effects

1. [Home](#) 2. [About Us](#) 3. [Contact Us](#) 4. [Privacy Policy](#)

Quality Attributes

- Reliability
- Availability
- Performance
- Scalability
- Security
- Maintainability
- Portability

100

Re
Instead
Apac
Ratib
Geog
Ther

- **Problems**
 - Per **technical** bet **problem** **sub**
 - Our
 - Con not
 - No pro

- pro

Week 4

Lecture 3 — Modern Distributed Architectures

How do we build large distributed systems?

Topics covered:

- Scale
- Reliability
- Performance
- Complexity
- Modularity
- Deployment
- Observability
- Security
- Cost

Key idea of Lecture 3: Modern systems are distributed systems, and distributed systems are complex.

Lecture 4 — Architectural Patterns

What are architectural patterns?

Patterns are reusable solutions to common problems.

Patterns are organized into groups:

- Client-Server
- Peer-to-Peer
- Microservices
- Event-Driven
- Service-Oriented
- Domain-Driven
- Functional
- Modular
- Scalable
- Reliable
- Secure
- Compliant
- Cost-Effective

Key idea of Lecture 4: Architectural patterns are reusable solutions to common problems.

this is the lec. week topic

In simple words:

An architectural pattern is a **general reusable solution for a common architecture problem.**

It is not a finished design, but a **template** that architects can adapt to different systems.

the relationship between three things:

challenge in that context.

minimize UI, logic, and data
distribute distributed services
components communicate.

our issues? → Solu.

In Idea

Architectural patterns help architects **reuse proven solutions** instead of designing everything from scratch.

side:

print for system organization
common vocabulary for architects
reference for solving recurring architecture problems.

important

growth

evolution

architecture

organization

pattern.

Architectural patterns and

Design Patterns

Design Patterns

Common Attributes

Context / Scope

Design Goals

Example Scenarios, Overview

Conclusion, while design patterns solve programming

Quality Attributes (from highlighted ones)

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Quality Attributes

Architectural Patterns vs Architectural Styles

this is something professors often ask in exams.

Architectural Styles	Architectural Patterns
Describe structure of components and connectors	Describe how to implement the architecture
Focus on "what the system looks like"	Focus on "how the system should be built"
More general	More domain-specific

Example:

- Client-Server — architectural style
- MVC — architectural pattern

The slides describe styles as the "what" and patterns as the "how".

	Architectural Patterns	Software Design Patterns
Definition	High-level structural solutions that define the overall system organization	Reusable solutions to common problems in software design at the code level
Scope	Large-scale (entire system or subsystems)	Small-scale (class or object level)
Focus	System structure, component relationships, communication	Object interaction, class structure, responsibilities
Goal	Ensure scalability, performance, reliability, and maintainability	Improve code reusability, flexibility, and maintainability
Impact of Change	Affects the entire application structure	Affects specific classes or modules

Key Quality Benefit

The main quality improvements of this pattern are:

- Modifiability
- Testability
- Maintainability

Because the system separates:

- Plain text
- Display
- Logic
- State

So changing the UI does not affect logic, and changing logic does not affect stored data.

Important Exam Idea

Architectural patterns provide:

Reusable high-level solutions for recurring architectural problems.

They operate at the **system architecture level**, not the code level.

The arch. pat. are 4 groups:

Summary of Lec 1,2,3,

Lecture 1 — Software Architecture Fundamentals

Other Software Architectures

Architecture	Description	Example
Client-Server	Centralized architecture where clients request services from a central server.	Web browser, database server
Peer-to-Peer	Decentralized architecture where all nodes are equal and can act as both clients and servers.	BitTorrent, blockchain
Microservices	Architecture where applications are built as a collection of small services that communicate over well-defined APIs.	Amazon, Netflix

Architecture vs Design vs Implementation

Level	Description	Example
Architecture	High-level system structure	Microservices vs monolith
Design	Low-level system design	Class diagram, database schema
Implementation	Actual code	Java, Python, C++

Architecture Elements

Element	Description	Example
Component	Reusable software module	Library, module
Interface	Contract between components	API, REST endpoint
Connector	Link between components	HTTP, message queue

Architecture Decisions

Type	Description
Strategic decisions	System architecture
Tactical decisions	Component design
Operational decisions	Deployment, infrastructure

Architectural decisions are hard to change later, so they must be made carefully.

Architectural Views

Different views describe architecture from different perspectives.

View	Purpose
Logical view	System functionality
Development view	Code organization
Process view	Runtime behavior
Physical view	Deployment site

These views help different stakeholders understand the system.

Architecture Documentation

Document	Purpose
Requirements	Define system requirements
Architecture	Define system architecture
Design	Define system design
Implementation	Define system implementation

Key Takeaways (Lecture 1)

Key Idea	Explanation
Architecture is a blueprint for a system.	It defines the structure and behavior of a system.
Architecture impacts quality attributes.	Performance, scalability, maintainability.
Architecture is a design decision.	It is a choice that affects the system's future.
Architecture is a communication tool.	It helps stakeholders understand the system.

Lecture 2 — Architectural Styles

What is an Architectural Style

Definition	Description
Definition	An architectural style defines a family of systems in terms of components, connectors, and constraints.
Purpose	Provide standard ways to organize software systems.
Focus	System structure and interaction between components.
Result	Reuse proven architectural solutions.

Basic Properties of Architectural Styles

Property	Description
Reusability	Can be reused in multiple systems.
Modifiability	Can be modified to meet new requirements.
Testability	Can be tested to ensure correctness.
Reliability	Can be relied upon to perform correctly.

Benefits of Using Architectural Styles

Benefit	Description
Reusability	Reuse proven architectural solutions.
Modifiability	Can be modified to meet new requirements.
Testability	Can be tested to ensure correctness.
Reliability	Can be relied upon to perform correctly.

Architectural Styles Overview

Style	Core Idea
Monolithic	Entire system in one unit
Layered	System divided into horizontal layers
Client-Server	Clients request services from servers
Pipe & Filter	Data flows through processing stages
Component-Based	System built from reusable components
Event-Driven	System reacts to external events
Plugin/Modular	Core system extended with plugins
Blackboard	Components collaborate through shared knowledge base
Peer-to-Peer	Components interact as equals

Pipe & Filter Architecture

Aspect	Description
Structure	Data flows through processing stages
Components	Filters (processors), pipes (data channels)
Advantages	Reusable processing components
Disadvantages	Difficult state sharing
Quality Attributes	Good scalability and modifiability

Example: Input → Filter1 → Filter2 → Filter3 → Output

Component-Based Architecture

Aspect	Description
Structure	System built from reusable components
Communication	Defined interfaces
Advantages	Reusability and modifiability
Disadvantages	Integration complexity
Quality Attributes	High maintainability and modifiability

Hexagonal (Ports & Adapters)

Aspect	Description
Structure	Isolate business logic from external systems
Components	Core logic, ports, adapters
Advantages	easier testing and flexibility
Disadvantages	more architectural complexity
Quality Attributes	strong maintainability

Event-Driven Architecture

Aspect	Description
Structure	Components communicate via events
Components	event producers, consumers
Advantages	loose coupling
Disadvantages	complex debugging
Quality Attributes	high scalability

Peer-to-Peer

Aspect	Description
Structure	Isolate business logic from external systems
Components	Core logic, ports, adapters
Advantages	easier testing and flexibility
Disadvantages	more architectural complexity
Quality Attributes	strong maintainability

Lecture 3 — Distributed Systems & Service Architectures

Service-Oriented Architecture (SOA)

Aspect	Description
Definition	Architecture where system functionality is provided as independent services communicating over a network.
Components	Services, service registry, message bus.
Communication	Service interfaces (e.g., SOAP, REST).
Goal	Reuse services across applications.

REST Architecture

Aspect	Description
Definition	Architectural style for distributed systems using stateless communication over HTTP.
Key Idea	Resources identified by URIs.
Communication	HTTP methods.

REST Operations

Operation	HTTP Method	Purpose
Read	GET	retrieve resource
Create	POST	create resource
Update	PUT	update resource
Delete	DELETE	remove resource

Advantages / Disadvantages

Advantages	Disadvantages
Service reuse	Complex infrastructure
Integration between systems	Slower communication
Loose coupling	Governance challenges

Quality Attributes

Attribute	Impact
Scalability	good
Modifiability	moderate
Reliability	good

Microservices

Aspect	Description
Definition	Architecture style for distributed systems using stateless communication over HTTP.
Key Idea	Resources identified by URIs.
Communication	HTTP methods.

Key Principles

Principle	Description
Decoupled	Decoupled components
Independent	Independent components
Loose coupling	Loose coupling

Advantages

Advantages	Disadvantages
Scalability	Complex infrastructure
Modifiability	Slower communication
Reliability	Governance challenges

Quality Attributes

Attribute	Impact
Scalability	high
Performance	good
Modifiability	good

Distributed Monolith (Anti-pattern)

Aspect	Description
Definition	System appears distributed but services are tightly coupled like a monolith.
Problem	Services cannot operate independently.
Cause	Poor microservices design.

Key Problems

Problem	Explanation
Lack of isolation	Services share dependencies.
Lack of independence	Services must deploy together.
Distributed complexity	Network overhead without benefits.

Migrating a Monolith

Steps to transform a monolithic system into microservices.

Step	Description
Identify bounded contexts	determine business domains
Create service boundaries	define service responsibilities
Extract services	move functionality out of monolith
Refactor requests	gradually shift traffic
Refactor monolith	remove old dependencies

- ② Component :
- Interoperability
 - Directory Pattern :
- ③ Events :
- Event Monitor
 - Publish - Subscriber
 - Messaging Bridge
 - Half Sync / Half Async
- ④ Composition :
- Scatter - Gather
 - Load Balancing
 - Orchestration
 - Choreography
 - Saga pattern
 - Map - Reduce

⑤ Layered :

separate layers, and

es.

like data structures) obtain.

Main Idea

Instead of mixing everything together

- Plan text
- UI = Data + Data to user phone

Layered pattern separates them

- Plan text
- Display Layer
- Logic Layer
- Storage Layer

Each layer handles different responsibility

Why Layered Patterns Are Used

They improve:

Quality Attribute	Why
Maintainability	easier to modify developer
Modifiability	UI can change without changing logic
Reusability	logic can be reused with different UIs
Testability	layers can be tested separately

- Display
- 3 components
- Display
2. Model - View - Controller
- separates an app into 3 components
- Model
 - View
 - Controller

Basic structure

- Plan text
- Model
- Controller
- Model
- View

Or conceptually

- Plan text
- Controller - Model
- Model - View
- View - User

Components Explained

Model

Responsible for:

- application data
- business logic
- system state

Examples:

- product database
- user accounts
- settings
- business rules

View

Responsible for:

- displaying information to users

Examples:

- web pages
- mobile screens
- dashboards

The view reads data from the model.

Controller

Responsible for:

- handling user input
- operating the model
- updating what's shown to display

Example actions:

- click, touch, drag, scroll, etc.

Diagram:

Controller (User Input) → View (Display) → Model (Data) → Controller (User Input)

Key Concepts

pattern focuses on:

- separation of concerns
- isolating system data
- isolating presentation layer
- controlling how logic interacts with state

related to MVC

also in an early version of MVC.

by introducing a controller to manage interaction.

Advantages

Advantage	Explanation
Clear separation	UI, logic, and control are separated
Reusable model	same data used in multiple views
Reusable UI	different interfaces for same system
Easier development	teams can work independently

Disadvantages

Disadvantage	Explanation
Increased complexity	more components
Learning curve	harder for beginners
Controller logic may grow large	if not managed well

Quality Attributes (Highlighted Ones)

Quality Attribute	Impact	Explanation
Availability	Neutral	MVC structure does not directly impact system availability
Reliability	Neutral	Depends on implementation rather than architecture
Performance	Slight overhead	communication between model/view/controller adds small overhead
Deployability	Neutral	Deployment depends on system infrastructure
Scalability	Neutral	MVC itself does not define scaling strategy
Modifiability	High	UI and logic can change independently
Security	High	model and controller can be tested separately
Maintainability	High	clear separation of responsibilities simplifies maintenance

Suitable Applications

MVC is commonly used in:

- web applications
- GUI systems
- mobile apps
- game interfaces

Examples:

- Django
- Ruby on Rails
- Spring MVC
- ASP.NET MVC
- iOS MVC frameworks

Real-World Example

Online shopping website:

MVC Components:

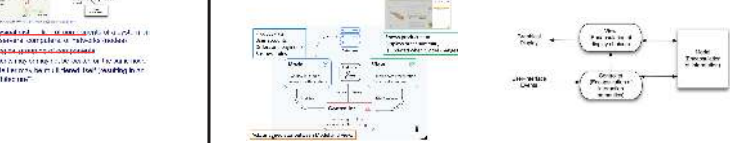
- Model: product database
- View: product page UI
- Controller: handles add-to-cart action

Compared to State-Logic-Display

State-Logic-Display:

- users
- interface
- arbitrary separation of data, logic, display
- UI
- arbitrary logic and data separation

MVC is basically a more structured version of State-Logic-Display



Layered Group Summary

Layered group patterns:

- State-Logic-Display
- Model-View-Controller (MVC)

Both focus on separating system responsibilities into layers.

② Component :

Main Idea

Component patterns help solve problems like:

- how components find each other
- how systems communicate across technologies
- how distributed services coordinate

Example:

- Plan text
- Service A - Service B - Service C

But these services may be:

Patterns in the Component Group

From the lecture, this group includes:

- Interoperability Pattern
- Directory Pattern

Quality Attributes	
Attribute	Impact
Scalability	poor
Maintainability	poor
Testability	reduced

Migration Strategy	
Strategy	Description
Strangler Fig Pattern	gradually replace monolith with services

Challenges of Distributed Systems

Challenge	Explanation
network latency	slower communication
fault tolerance	systems must handle failures
data consistency	difficult across services
monitoring	harder to observe system state

Lecture 3 Key Takeaways

Key Idea

Modern systems are distributed

Microservices enable scalability

Poor microservices design leads to distributed monolith

Distributed systems introduce complexity

Migration from monolith should be gradual

Lecture 4 — Architectural Patterns & Anti-Patterns

Architectural Patterns

Concept	Explanation
Abstraction	Provides a pattern or reusable solution to common architectural problems
Pattern	Typical architectural components
Structure	Design pattern describes system, problem, and solution
Template	Helps architects reuse proven system structures

Styles vs Patterns vs Design Patterns

Concept	Style	Pattern
Architectural Style	Overall system structure	Interoperability
Architectural Pattern	Problem and solution	MVC
Design Pattern	Code-level solution	Singleton

Pattern Groups in the Lecture

Pattern Group	Purpose
Layered Patterns	separate responsibilities into layers
Component Patterns	manage communication between components
Event Patterns	systems react to events
Composition Patterns	coordinate multiple services

Model-View-Controller (MVC)

Component	Role
Model	application data and business logic
View	user interfaces
Controller	manages interaction between model and view

Advantages / Disadvantages

Advantages	Disadvantages
separation of concerns	more components
multiple views of same data	complex controllers

Quality Attributes

Attribute	Impact
Modifiability	high
Maintainability	high
Testability	good

Component Patterns

Interoperability Pattern

Aspect: Description

Purpose: allow systems built with different technologies to communicate

Tools: APIs, middleware, messaging systems

Quality Attributes

Attribute	Impact
Deployability	high
Scalability	good
Modifiability	high

Event Patterns

Event Monitor

Aspect: Description

Issue: system monitors events and notifies clients

Example: monitoring tools

Publish-Subscribe

Aspect: Description

Issue: subscribers are notified, rather than request data

Example: message brokers, event-driven architectures

Messaging Bridge

Aspect: Description

Issue: loosely-coupled messaging infrastructure

Example: message brokers, event-driven architectures

Half-Sync / Half-Async

Aspect: Description

Issue: combine synchronous request handling with asynchronous processing

Example: async request processing, async request processing

Composition Patterns

Scatter-Gather

Aspect: Description

Issue: send request to multiple services and combine results

Example: parallel processing

Load Balancing

Aspect: Description

Issue: distribute requests across multiple servers

Example: improve performance and availability

Orchestration

Aspect: Description

Issue: central controller manages workflow between services

Choreography

Aspect: Description

Issue: services react to events without central controller

Saga Pattern

Aspect: Description

Issue: manage distributed transactions using local transactions and compensation

Example: Order -> Payment -> Shipping

If failure occurs: Cancel payment -> Cancel order

Map-Reduce

Aspect: Description

Issue: process large datasets using distributed parallel computation

Example: map -> shuffle -> reduce

Lecture 4 Key Takeaways

Key Idea

Architectural patterns provide reusable architecture solutions

Patterns are grouped by problem type

Anti-patterns represent bad design practices

Good architecture improves quality attributes

Explanation

MVC, Saga, MapReduce

layered, component, event, composition

should be avoided

scalability, maintainability

Summary of Sum.

Ultra-Short Memory Version (Exam Recall)

Lecture	5-Word Memory Cue
Lecture 1	Why architecture exists
Lecture 2	Ways to structure systems
Lecture 3	How distributed systems work
Lecture 4	Solutions and mistakes in architecture

Concept Flow of the Course

Lecture 1	Architecture fundamentals
Lecture 2	Architecture structures (styles)
Lecture 3	Modern distributed architectures

What This Helps With In the

This table helps you instantly answer questions

Question

What is software architecture?

What architecture structures exist?

How are modern distributed systems built?

How do we solve architectural problems?

2- Directory

Definition
The Directory Pattern allows components or services in a distributed system to find each other dynamically without knowing their exact location.

From the lecture:
It provides location transparency, ensuring components can discover services without knowing their physical address.
In simple words:
Instead of hardcoding service addresses, systems ask a central directory where the service is located.

Topology (Structure)

Service Registry (Directory)
A central registry storing:
• service names
• service locations
• service health status
Examples:
• DNS
• service registry
• discovery server

Client
The system requesting a service.
Example:
• web application
• microservice
• API gateway

Service Instance
The actual service providing functionality.
Example:
• request service
• authentication service
• order processor

Two Types of Directory Pattern

Client-Side Discovery
The client directly queries the service registry.

Server-Side Discovery
The client sends requests to a load balancer or gateway, which handles discovery.

Main Idea
The directory pattern removes hardcoded service locations.

Advantages
• Location transparency
• Dynamic service discovery
• Flexibility
• Improved scalability

Disadvantages
• Extra infrastructure
• Single point of failure
• Additional network communication

Quality Attributes (Highlighted Ones)

Suitable Applications
Directory pattern is widely used in:
• microservices architecture
• cloud systems
• distributed applications
• service-oriented architectures

Real-World Example
Example: DNS (Domain Name System)
Instead of remembering IP addresses, you use domain names like google.com.

Software Architecture — Key Differences Between Lectures (1-4)

Lecture	Core Focus	Main Question Answered	Key Concepts	Architecture Level	Example Topics	One
Lecture 1	Fundamentals of Software Architecture	Why architecture matters and what it consists of	components, connectors, quality attributes, stakeholders, architectural decisions	Conceptual foundation	quality attributes, architectural views, trade-offs	Arch
Lecture 2	Architectural Styles	What structural patterns can organize a system	components, connectors, topology, constraints	Structural system design	layered, client-server, pipe-filter, event-driven, P2P	Arch
Lecture 3	Distributed Architectures	How modern large systems are built and deployed	services, distributed systems, microservices, REST, system migration	System architecture evolution	SOA, REST, microservices, distributed monolith, migration patterns	Mod
Lecture 4	Architectural Patterns & Anti-Patterns	How to solve architectural problems and avoid bad designs	reusable patterns, event systems, service coordination, architecture mistakes	Architectural problem-solving	MVC, Saga, MapReduce, orchestration, anti-patterns	Arch

Software Architecture — Professor Trap Sheet

Definitions (Very Common Questions)

Question	Expected Answer
What is Software Architecture?	The high-level structure of a software system, including components, connectors, and architectural decisions that achieve system quality attributes.
What is an Architectural Style?	A family of systems defined by components, connectors, topology, and constraints that determine how systems are organized.
What is an Architectural Pattern?	A reusable solution to a recurring architectural problem in a specific context.
Difference between architectural style and pattern.	Style defines system structure, patterns provide solutions to architectural problems.
Difference between architectural patterns and design patterns.	Architectural patterns apply to system architecture, design patterns apply to class-level design.

Quality Attributes (Almost Always Asked)

Attribute	Meaning
Availability	system operational uptime
Reliability	system works without failure
Performance	response time and throughput
Scalability	ability to handle increased workload
Deployability	ease of deploying system
Modifiability	ease of changing functionality
Testability	ease of testing system components
Maintainability	ease of maintaining system

REST Questions

Question	Answer
What is REST?	architectural style for stateless web services using HTTP.
REST operations	GET, POST, PUT, DELETE
REST principle	stateless communication

Distributed System Questions

Question	Answer
What is a distributed monolith?	system appears distributed but services are tightly coupled like a monolith.
Why distributed systems are complex	network latency, failures, consistency challenges
Migration pattern from monolith	Strangler Fig pattern

Pattern Recognition Questions

Pattern	Key
MVC	separation of concerns
Directory Pattern	service discovery
Publish-Subscribe	asynchronous communication
Scatter-Gather	distributed processing
Load Balancing	distributed load
Orchestration	centralized control
Choreography	decentralized control
Saga	distributed transaction
MapReduce	distributed data processing

Comparison Questions

Orchestration vs Choreography

Feature	Orchestration	Choreography
Control	centralized	decentralized
Workflow	controlled by orchestrator	emerged from interactions
Scalability	moderate	high
Monitoring	easier	harder

Anti-Patterns (Very Common Short Questions)

Anti-pattern	Meaning
Spaghetti System	ad hoc integrations between systems
Spaghetti Enterprise	isolated enterprise systems
Autogenerated Spaghetti	legacy monolith wrapped as services
Ball of Mud	chaotic architecture
Architecture by Implication	architecture not explicitly defined
Reinvent the Wheel	building existing solutions
Vendor Lock-in	dependence on specific vendor technology

Pattern Groups (Lecture 4)

Group	Purpose	Patterns
Layered	separate responsibilities	Stratified Logic Display, MVC
Component	communication between components	Interoperability, Directory
Event	event-based communication	Event Monitor, Publish, Messaging Bridge
Composition	coordinate multiple services	Scatter-Gather, Load Balancing, Saga

3- events : Notification Patterns

Components react to events instead of calling each other.

Main Idea
Instead of direct communication, components react to events.

Example
Payment Service, Shipping Service, Notification Service.
Each component subscribes to events and responds when needed.

2- Publish - subscribe

Components react to events instead of calling each other.

Main Idea
Components react to events instead of calling each other.

3- Messaging Bridge

Used to connect two different messaging systems or messaging buses that are not directly compatible.

Main Idea
Add a layer hiding asynchronous interactions behind a synchronous interface.

2- Half-Sync / half-Async

Separates a sync to 2 parts:
• as sync that handles incoming req.
• as async that processes tasks sequentially.

2- Publish - subscribe

Components react to events instead of calling each other.

Main Idea
Components react to events instead of calling each other.

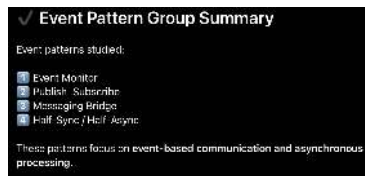
3- Messaging Bridge

Used to connect two different messaging systems or messaging buses that are not directly compatible.

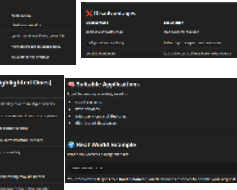
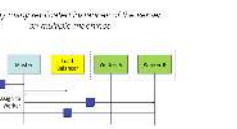
Main Idea
Add a layer hiding asynchronous interactions behind a synchronous interface.

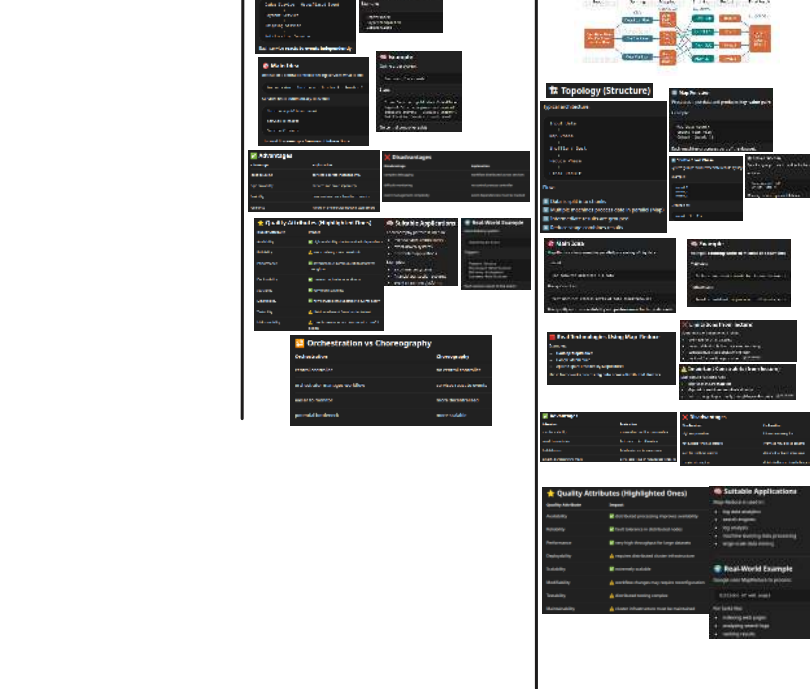
2- Half-Sync / half-Async

Separates a sync to 2 parts:
• as sync that handles incoming req.
• as async that processes tasks sequentially.



or components are combined together to





Architectural Anti-Patterns

usually causes problems instead of solving
solution to a problem that generates negative
and architecture practices.

Why Anti-Patterns Happen

Anti-patterns usually occur when:

- architects lack experience
- good patterns are used in the wrong context
- architecture is not planned properly
- systems evolve without structure

- 1- Stovepipe System
 - 2- Stovepipe Enterprise
 - 3- Autogenerated Stovepipe
 - 4- Jumble (Ball of Mud)
 - 5- Architecture by Implication
 - 6- Reinvent the Wheel
 - 7- Vendor Lock-in
- SA - Lecture 4

Enterprise

3- Auto generated stovepipe

4- Jumble (Ball of mud)



3- Auto generated stovepipe

Definition: The system is composed of many small, self-contained components that are not designed to work together. Each component is a 'stovepipe' that only communicates with its own internal components.

Typical Structure:

- Component A
- Component B
- Component C
- Component D
- Component E
- Component F
- Component G
- Component H
- Component I
- Component J
- Component K
- Component L
- Component M
- Component N
- Component O
- Component P
- Component Q
- Component R
- Component S
- Component T
- Component U
- Component V
- Component W
- Component X
- Component Y
- Component Z

Negative Effects:

- High technical debt
- Difficult to maintain
- Difficult to debug
- Difficult to test
- Difficult to deploy
- Difficult to scale
- Difficult to integrate
- Difficult to upgrade
- Difficult to replace
- Difficult to remove
- Difficult to add
- Difficult to change
- Difficult to evolve
- Difficult to adapt
- Difficult to respond
- Difficult to recover
- Difficult to survive
- Difficult to thrive
- Difficult to prosper
- Difficult to flourish
- Difficult to blossom
- Difficult to bloom
- Difficult to fruit
- Difficult to seed
- Difficult to sow
- Difficult to plant
- Difficult to grow
- Difficult to mature
- Difficult to age
- Difficult to decline
- Difficult to fall
- Difficult to die
- Difficult to resurrect
- Difficult to reborn
- Difficult to regenerate
- Difficult to rejuvenate
- Difficult to refresh
- Difficult to renew
- Difficult to restore
- Difficult to revive
- Difficult to resurrect
- Difficult to reborn
- Difficult to regenerate
- Difficult to rejuvenate
- Difficult to refresh
- Difficult to renew
- Difficult to restore
- Difficult to revive

Real-World Example:

ebay

ebay's architecture is a classic example of an autogenerated stovepipe. It consists of many small, self-contained components that are not designed to work together. Each component is a 'stovepipe' that only communicates with its own internal components.

4- Jumble (Ball of mud)

Definition: The system is a chaotic mess of code and components that are not organized in any way. It is a 'ball of mud' that is impossible to manage.

Typical Structure:

- Component A
- Component B
- Component C
- Component D
- Component E
- Component F
- Component G
- Component H
- Component I
- Component J
- Component K
- Component L
- Component M
- Component N
- Component O
- Component P
- Component Q
- Component R
- Component S
- Component T
- Component U
- Component V
- Component W
- Component X
- Component Y
- Component Z

Negative Effects:

- High technical debt
- Difficult to maintain
- Difficult to debug
- Difficult to test
- Difficult to deploy
- Difficult to scale
- Difficult to integrate
- Difficult to upgrade
- Difficult to replace
- Difficult to remove
- Difficult to add
- Difficult to change
- Difficult to evolve
- Difficult to adapt
- Difficult to respond
- Difficult to recover
- Difficult to survive
- Difficult to thrive
- Difficult to prosper
- Difficult to flourish
- Difficult to blossom
- Difficult to bloom
- Difficult to fruit
- Difficult to seed
- Difficult to sow
- Difficult to plant
- Difficult to grow
- Difficult to mature
- Difficult to age
- Difficult to decline
- Difficult to fall
- Difficult to die
- Difficult to resurrect
- Difficult to reborn
- Difficult to regenerate
- Difficult to rejuvenate
- Difficult to refresh
- Difficult to renew
- Difficult to restore
- Difficult to revive

Real-World Example:

ebay

ebay's architecture is a classic example of a jumble. It is a chaotic mess of code and components that are not organized in any way. It is a 'ball of mud' that is impossible to manage.

- Problem
 - horizontal and vertical design elements are intermixed.
 - The result is an unstable and unmanageable system.
- Solution
 - The first step is to identify the horizontal design elements and delegate them to a separate architecture.

- Then use the horizontal elements to capture the common interoperability functionality in the architecture.

7-vendor lock-in

Example Example is software development Distinct of using an existing logging library, a developer builds a custom logging system. Problems appear: • missing features • bugs in logging logic • maintenance cost faced When a stable library already exists	Example Example is software development Distinct of using an existing logging library, a developer builds a custom logging system. Problems appear: • missing features • bugs in logging logic • maintenance cost faced When a stable library already exists
--	--

Reinvent the Wheel 

[illegible]

of building a custom distributed messaging system, companies use:

the Kafka
tion
ple Pub/Sub

systems are already tested, scalable, and reliable

• *Solution*

- Design knowledge buried in legacy assets can be leveraged to reduce time-to-market, cost, and risk.
- Many ready to use products that are well tested.

Skellam

- Skellam's upper triangle

Skellam's lower triangle

Skellam's lower triangle

Skellam

Skellam's lower triangle

Skellam's lower triangle

[illegible]

Negative Effects	
Problem	Explanation
1-100% Reliability	system tied to one vendor
for migration	switching is difficult to even reconfigure
reduced maintainability power	vendor controls platform
technology dependency	system version and application dependent

★ Quality Attributes Impact	
Quality Attribute	Impact
Availability	▲ decrease need to restore system state
Reliability	▲ reduce restoration platform effort, duration
Performance	▲ limited restoration duration
Flexibility	✗ decelerate, limited to various circumstances
Scalability	▲ fast to scale capabilities
Usability	✗ introduce a constrained user interface
Portability	▲ enhance flexibility, testing and deployment
Recoverability	▲ system state based on latest available technology

[illegible]

Stakeholders in Software Architecture	
Stakeholder	Interest
Developers	system structure and implementation
Architects	design decisions and trade-offs
Program managers	cost and time-to-market
Users	usability and performance
Operators	deployment and maintenance

Architecture Trade-offs	
Architecture trade-offs are competing qualities.	
Example	
Trade-off	Example
Performance vs. Maintainability	extensive front-end checks vs. maintainable
Flexibility vs. Efficiency	strongly-typed languages vs. flexibility
Scalability vs. Consistency	distributed systems vs. consistency
Architecture often involves choosing the best compromise.	

Architectural Styles and Quality Attributes	
Quality Attribute	Impact of Architecture
Performance	depends on control plane overhead
Scalability	distributed architectures scale better
Reliability	redundant systems increase reliability
Maintainability	modular architectures increase maintainability
Modifiability	well-encapsulated systems increase modifiability

Client-Server Architecture	
Aspect	Description
Structure	clients request services from server
Components	clients, server
Advantages	centralized control
Disadvantages	server bottleneck
Quality Attributes	good maintainability but limited scalability
Example	
Client → Server → Database	

Plugin / Microkernel Architecture	
Aspect	Description
Structure	microkernel + plugins
Core	base system functionality
Plugins	extend system capabilities
Advantages	modifiability
Disadvantages	plugin management complexity
Quality Attributes	high modifiability
Example: IoT plugins	

Peer-to-Peer Architecture	
Description	
	decentralized nodes
	each node acts as client and server
	high scalability and resilience
	security and coordination challenges
	strong scalability

Services Architecture	
Description	
	System composed of small independent services , each responsible for a specific business capability
	REST APIs or messaging
	Independent deployments for each service
Messaging	
	services represent a specific domain
	services developed and deployed separately
	minimal dependency between services
Pros / Disadvantages	
	Disadvantages
	distributed complexity
Deployment	difficult debugging
Scalability	network overhead
Quality Attributes	
	Impact
	very high
	high
	very high
	difficult

Fallacies of Distributed Computing	
Common false assumptions when designing distributed systems.	
Fallacy	Reality
Network is reliable	networks fail
Latency is zero	network delays exist
Bandwidth is infinite	bandwidth limited
Network is secure	security risks exist
Topology doesn't change	infrastructure evolves
One administrator	multiple control domains

	Explanation
	services communicate over networks
	independent service architecture
s	architecture must be planned carefully
	networking, failures, coordination
	use patterns like Strangler Fig

Behavioral Patterns	
Composite-Display Pattern	
description	- separate options into state, logic, and display - display → logic → state - separate parts that change in different ways
Pros / Disadvantages	
pros	flexible and elegant
cons	- additional complexity - increases class methods
Attributes	
scope	project
priority	high
frequency	high
importance	high

Query Pattern	Description
discovery	services dynamically discover each other services registry or directory
discovery	client queries registry
discovery	load balances peer-to-peer lookup
Attributes	Impact
complexity	high
performance	good

Architectural Anti-Patterns	
Assignment: Create half and Android products	
And business	Deadline
Develop system	Uncontrolled, ad hoc module integrations
Services integration	Isolated customer access organization
Is implemented: Overview	Empire resources assigned on a whim
Combination of	Shared system security
Architecture by design	Not covered by a test security
Generalized Method	Uncontrolled system integration
Generalized Method	Uncontrolled system integration
Generalized Method	Uncontrolled system integration

tion

Which Lecture
Lecture 1
Lecture 2
Lecture 3
Lecture 4

-Sentence Summary

Architecture defines the high-level structure of a system and determines how design decisions achieve quality attributes such as performance, scalability, and maintainability.

Architectural styles define common structural ways to organize system components and communication to achieve specific system qualities.

Modern software systems are distributed collections of services that communicate over networks, requiring careful design to manage scalability, reliability, and complexity.

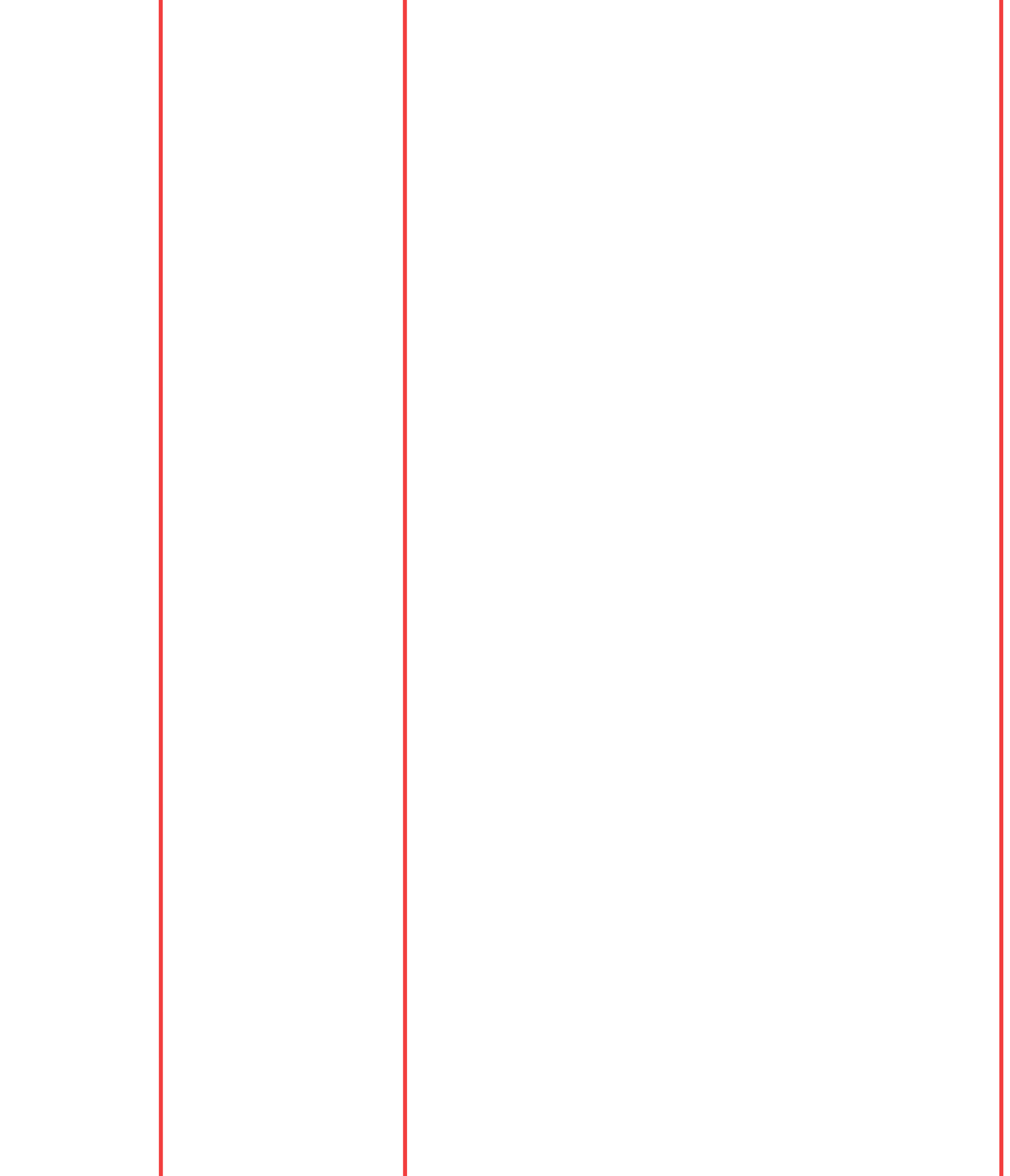
Architectural patterns provide reusable solutions for common system design problems, while anti-patterns describe common architectural mistakes to avoid.

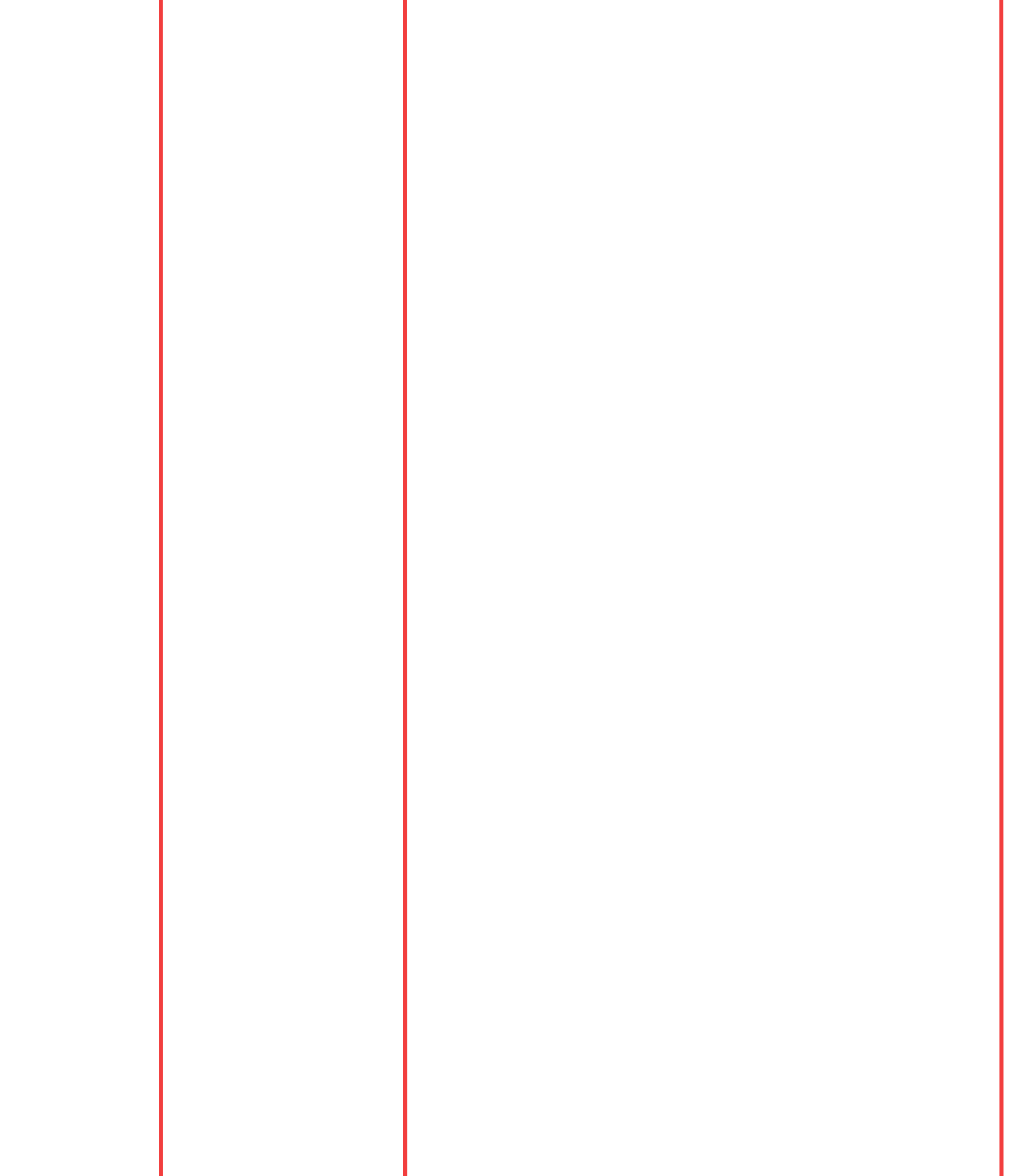
Identification Questions	
Key idea	Typical example
single large application	traditional enterprise apps
system divided into layers	web apps
clients require server services	web systems
data processed in stages	compilers
reusable components	enterprise software
systems react to events	messaging systems
decentralized nodes	Bitcoin

Services Questions	
	Answer
Is it a Service Architecture?	system composed of small independent services communicating through APIs
	bounded context and service independence
	scalability, independent deployment
Challenges	distributed complexity, debugging difficulty

- Questions**
- Idea**
 - separate model, view, controller
 - services discover each other dynamically
 - publishers send events, subscribers receive them
 - route requests to multiple services and aggregate results
 - route requests across servers
 - central controller manages workflow
 - services coordinate through events
 - distributed transactions with compensation
 - distributed big data processing

(Very Common)	
Choreography	
decentralized	
event-driven	
high	
harder	





1

2

3